

03 - Analisi Architettuale

Alessandro Gardini

`alessandro.gardini7@studio.unibo.it`

Lorenzo Rossi

`lorenzo.rossi50@studio.unibo.it`

Tirocizio - Università di Bologna - Campus di Cesena

10 giugno 2026

1 Correzione Analisi del Dominio

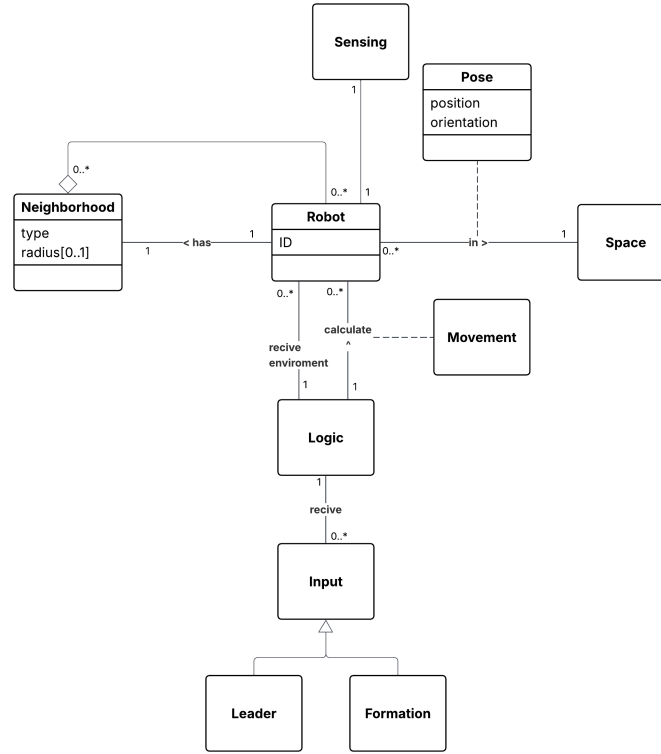


Figura 1: Analisi del dominio corretta

L'oggetto *Logic* rappresenta la funzione di decisione del sistema, ispirata al modello computazionale dell'*aggregate computing*, in cui ogni dispositivo calcola il proprio output in funzione del contesto locale e input che riguardano tutti i dispositivi. In questo schema, *Logic* riceve in ingresso il **contesto**, che racchiude lo stato corrente del robot e delle entità a esso associate, e gli **input**, modellati tramite la gerarchia *Input*. A partire da queste informazioni, *Logic* produce una **decisione**, ovvero:

$$Logic : (Context, Input) \rightarrow Decision$$

Abbiamo anche reificato in un'entità *Sensing* che rappresenta tutti i dati che il robot/sistema potrà valutare in futuro.

2 Analisi architetturale

2.1 Modulo Offloading Manager

Il modulo di offloading si pone l'obiettivo di organizzare la logica delle decisioni di offloading e di porsi come interfaccia per tutto ciò che riguarda l'offloading permettendo richieste come la variazione di responsabilità sul calcolo o informazioni riguardanti lo stato di affaticamento del sistema.

2.1.1 Comunicazioni verso i robot

Per la comunicazione verso i robot si è pensato ad un sistema nel quale i robot per iniziare la comunicazione effettuino una registrazione al offloading manager, segnalando la loro disponibilità ad operare in offloading, attraverso una richiesta `POST` nel quale specifichino il proprio identificativo e l'endpoint disponibile per le future richieste. Il manager predispone anche un endpoint che il robot può chiamare per segnalare la propria volontà di modificare il proprio stato di offloading, il manager considererà queste richieste in maniera diversa rispetto ad altre richieste esterne, avendo la sicurezza che il robot ha già approvato questo cambiamento.

2.1.2 Comunicazione verso altri moduli

La comunicazione per le richieste di offloading di altri moduli, escluso i robot, avviene attraverso un endpoint `POST` nel quale si specificano l'id del robot interessato e il tipo di offloading richiesto, infatti si prende in considerazione la possibilità di effettuare offloading parziali in cui solo una parte delle responsabilità viene presa dal sistema, come ad esempio, il calcolo dei vicini sul sistema e mantenere la posizione e lo spostamento sul robot. Il manager si occupa anche di fornire informazioni sullo stato del sistema relative all'offloading, con anche la possibilità di specificare un identificato, predisposto anche un possibile endpoint che possa restituire informazioni come il carico di lavoro del sistema o il totale di droni con capacità di offloading o che lo stanno attualmente effettuando.

2.2 Modulo Robot

Il modulo robot dovrebbe esporre endpoint per la comunicazione con l'offloading manager, si è pensato ad una richiesta `POST` nel quale body si specifichi l'identificatore del robot per permettere un controllo che il messaggio inviato sia corretto e il tipo di offloading richiesto, a questa richiesta si risponderà con un messaggio che confermi il cambio sia stato accettato e il tipo di offloading in cui si è passato per maggiore controllo.

2.3 Moduli Aggregate, Aruco e Neighborhood

Questi moduli vengono accorpati nella descrizione presentando un'organizzazione simile, tutti e tre presenteranno la possibilità di ottenere informazioni dagli endpoint descritti nel offloading manager ed esporranno endpoint `POST` di tipo start e stop attraverso i quali verrà passato un identificatore che rappresenterà il robot di cui si dovrà iniziare o smettere di calcolare le rispettive parti.

2.4 Modulo dashboard

Questo modulo effettuerà richieste di offloading e di informazioni sul sistema al offloading manager come sopra riportato.

2.5 Modulo di traduzione protocolli

Il modulo di traduzione protocolli funge da gateway tra il broker MQTT interno e i client esterni, esponendo le stesse informazioni attraverso protocolli diversi a seconda delle esigenze del consumatore. La logica di traduzione è unica: ogni messaggio ricevuto dal broker viene inoltrato a tutti i sottoscrittori registrati sul canale corrispondente, indipendentemente dal protocollo utilizzato.

Il modulo supporta attualmente HTTP e CoAP. WebSocket è stato escluso in quanto il broker Mosquitto espone nativamente un endpoint MQTT over WebSocket, rendendo superflua un'ulteriore implementazione in questo modulo. Per HTTP si è sfruttata la generazione automatica della documentazione offerta da FastAPI tramite OpenAPI. Per CoAP la documentazione è stata redatta manualmente in formato Markdown.

2.5.1 Considerazioni per *HTTP*

Per HTTP si è scelto il pattern *webhook* (o callback). Il client effettua una richiesta **POST** a un endpoint di sottoscrizione fornendo il proprio URL di callback, e il server inoltra gli aggiornamenti a quell'URL non appena arrivano dal broker MQTT. Questo approccio è stato documentato tramite OpenAPI, sfruttando la funzionalità *callbacks* di FastAPI che permette di descrivere formalmente la forma della richiesta che il server effettuerà verso il client.

2.5.2 Considerazioni per *CoAP*

Per CoAP si è adottato il meccanismo *Observe* (RFC 7641), che è la soluzione nativa del protocollo per la notifica di aggiornamenti. Il client registra l'interesse per una risorsa includendo l'opzione **Observe: 0** in una richiesta **GET**; il server inoltra automaticamente ogni aggiornamento successivo senza che il client debba ripetere la richiesta. Le richieste **GET** prive dell'opzione *Observe* vengono rifiutate, poiché il server non dispone di un valore corrente da restituire.

2.5.3 Considerazioni

La sintassi degli URI rispecchia deliberatamente quella dei topic MQTT, incluso l'uso del carattere `+` come wildcard per sottoscrivere a tutti i robot contemporaneamente, sia in *HTTP* che in *CoAP*.